

Análisis crítico del modelo de consistencia implementado en el PFC (SCLI) y evaluación de alternativas:

¿Es el modelo elegido el óptimo para el dominio del sistema?

Iván Andrés Villamarin Cuenca
Facultad de Ciencias de la Computación
Universidad Técnica Estatal de Quevedo
Quevedo, Ecuador

Resumen—El presente informe analiza el modelo de consistencia y replicación utilizado en el Sistema de Control de Laboratorios Informáticos (SCLI), desarrollado mediante una arquitectura de microservicios y el patrón Database per Service. Se estudia cómo se mantienen correctos los datos dentro de cada servicio y qué problemas pueden aparecer cuando una operación depende de varios microservicios. El análisis identifica que las operaciones críticas, como la autenticación, los permisos y las reservas, requieren consistencia fuerte para evitar accesos incorrectos o cruces de horarios. En cambio, los reportes, las notificaciones y la auditoría pueden trabajar con consistencia eventual. También se comparan dos alternativas de replicación: PostgreSQL con arquitectura primaria-réplica y CockroachDB en un clúster multinodo. Se concluye que el modelo actual, basado en CockroachDB de un solo nodo, es adecuado para desarrollo, pero no es óptimo para producción porque no ofrece replicación real ni alta disponibilidad. La solución recomendada es aplicar un modelo híbrido según la importancia de cada operación.

Index Terms—consistencia, replicación, microservicios, CockroachDB, PostgreSQL, CAP, PACELC, Database per Service, alta disponibilidad, SCLI

I. INTRODUCCIÓN

El Sistema de Control de Laboratorios Informáticos (SCLI) es una plataforma web destinada a organizar el uso de los laboratorios de una institución educativa. El sistema permitirá gestionar usuarios, horarios, laboratorios, equipos, reservas, solicitudes, reportes, notificaciones y monitoreo.

La nueva versión del SCLI se desarrolla con una arquitectura de microservicios. Esto significa que el sistema se divide en varios servicios pequeños, donde cada uno cumple una función específica. Por ejemplo, existen servicios separados para autenticación, usuarios, laboratorios, reservas y solicitudes.

Cada microservicio tendrá su propia base de datos PostgreSQL, siguiendo el patrón Database per Service. Los servicios no compartirán tablas directamente, sino que intercambiarán información mediante APIs REST y mensajes enviados con RabbitMQ. Esta separación mejora el mantenimiento y permite desarrollar cada servicio de manera independiente,

pero también hace más difícil mantener los datos actualizados entre todos los componentes.

La consistencia de los datos es especialmente importante en las reservas. Por ejemplo, si dos usuarios intentan reservar el mismo laboratorio en el mismo horario, el sistema debe aceptar una sola reserva y rechazar la otra. También debe evitar que se reserve un laboratorio que se encuentre ocupado, cerrado o en mantenimiento.

No todos los datos requieren el mismo nivel de actualización. Las reservas, los permisos y la disponibilidad deben mantenerse correctos casi de inmediato. En cambio, los reportes, las notificaciones y algunos datos de monitoreo pueden actualizarse con un pequeño retraso sin afectar el funcionamiento principal.

La replicación consiste en mantener copias de una base de datos para reducir el riesgo de pérdida de información y mejorar la disponibilidad. La arquitectura actual define una base de datos independiente para cada microservicio, pero todavía es necesario determinar qué estrategia de replicación se implementará.

Por lo tanto, este informe analizará el modelo de consistencia y la estrategia de replicación más adecuados para el SCLI. También se compararán diferentes alternativas para determinar cuál ofrece el mejor equilibrio entre datos correctos, disponibilidad, velocidad y tolerancia a fallos.

II. MARCO TEÓRICO

II-A. Bases de datos distribuidas y microservicios

Una base de datos distribuida almacena o replica información en varios nodos conectados mediante una red. Su propósito es mejorar la disponibilidad, la tolerancia a fallos y la capacidad de atender más solicitudes. Sin embargo, cuando existen varias copias de un mismo dato, surge el problema de mantenerlas actualizadas y evitar que diferentes usuarios obtengan versiones distintas de la información [1].

La arquitectura de microservicios divide una aplicación en servicios pequeños y autónomos. Cada servicio puede

desarrollarse, desplegarse y escalarse de manera independiente, además de administrar su propia base de datos. La comunicación entre servicios se realiza normalmente mediante solicitudes HTTP o mensajes asíncronos. Esta independencia mejora el mantenimiento, pero dificulta el control de las operaciones que modifican datos almacenados en diferentes servicios [2].

Es importante diferenciar el patrón Database per Service de la replicación. Database per Service significa que cada microservicio administra datos diferentes y no accede directamente a las tablas de otros servicios. La replicación, en cambio, consiste en mantener varias copias del mismo conjunto de datos. Por lo tanto, las bases `auth_db`, `usuarios_db`, `academico_db`, `laboratorios_db` y `reservas_db` del SCLI no son réplicas entre sí, porque cada una almacena información perteneciente a un dominio distinto [1], [2].

Según la arquitectura descrita, el SCLI aplica Database per Service, pero todavía no se ha especificado una estrategia real de replicación para cada base PostgreSQL. Para afirmar que existe replicación sería necesario configurar, por ejemplo, un nodo principal y uno o varios nodos secundarios, un sistema multi-líder o un conjunto de nodos que trabajen mediante quórum. Esta distinción será importante cuando se analice el modelo realmente implementado.

II-B. Modelos de consistencia

Un modelo de consistencia establece las reglas que determinan cuándo una modificación puede ser observada por otros usuarios o nodos. En términos sencillos, indica qué tan actualizada debe estar una lectura después de realizarse una escritura. Los modelos no ofrecen las mismas garantías: algunos priorizan que todos vean inmediatamente el último dato, mientras que otros aceptan retrasos para mejorar la disponibilidad y el tiempo de respuesta [1].

La consistencia fuerte o linealizable busca que el sistema se comporte como si existiera una sola copia actualizada de los datos. Una operación confirmada debe ser visible para las lecturas posteriores y debe respetar el orden real en que ocurrió. Este modelo facilita el control de reglas críticas, pero requiere coordinación entre los nodos y puede aumentar el tiempo de respuesta [1], [4].

En el SCLI, este modelo sería apropiado para comprobar la disponibilidad y registrar una reserva. Si una persona confirma la reserva de un laboratorio, una segunda solicitud para el mismo laboratorio, fecha y horario debe encontrar el recurso ocupado. De lo contrario, el sistema podría aceptar dos reservas incompatibles. Esta aplicación al SCLI es una inferencia basada en las garantías de la consistencia fuerte descritas en [1] y [4].

La consistencia secuencial garantiza que todos los nodos observen las operaciones en el mismo orden, aunque ese orden no tenga que coincidir exactamente con el tiempo real. Por ejemplo, todos los usuarios podrían observar primero una reserva y después su cancelación, pero el sistema no garantiza que la actualización sea visible inmediatamente después de ejecutarse [1].

La consistencia causal mantiene el orden de las operaciones que tienen una relación de causa y efecto. Si una segunda acción depende de una primera, todos los nodos deben observarlas en ese orden. En cambio, las operaciones que no están relacionadas pueden procesarse en órdenes diferentes. Este modelo ofrece garantías superiores a la consistencia eventual sin exigir toda la coordinación de un modelo linealizable [1], [3].

Un ejemplo en el SCLI sería una solicitud de reserva seguida por su aprobación. La aprobación depende de que la solicitud exista, por lo que ningún servicio debería observar la aprobación antes de conocer la solicitud. La consistencia causal permitiría conservar ese orden lógico, aunque otros cambios independientes puedan propagarse de manera asíncrona.

La consistencia de lectura de los propios cambios, conocida como read-your-writes, garantiza que un usuario pueda observar los datos que acaba de registrar. Por ejemplo, después de crear una solicitud, el docente debería verla inmediatamente en su historial, aunque otros usuarios o réplicas todavía no hayan recibido la actualización [1].

La consistencia eventual permite que algunas copias muestren temporalmente información anterior. La garantía consiste en que, si dejan de producirse nuevas modificaciones, todas las réplicas terminarán convergiendo hacia el mismo estado. Este modelo reduce la coordinación y puede mejorar la disponibilidad, pero no debe utilizarse sin controles adicionales cuando una operación depende del valor más reciente [1], [4].

En el SCLI, la consistencia eventual podría utilizarse para actualizar reportes, enviar notificaciones o consolidar métricas de monitoreo. Un reporte puede tardar algunos segundos en mostrar una reserva reciente sin provocar un conflicto operativo. En cambio, utilizar únicamente consistencia eventual para confirmar la disponibilidad de un laboratorio podría producir reservas duplicadas.

De manera práctica, los modelos pueden entenderse como un espectro que va desde garantías fuertes hasta garantías más flexibles. La linealizabilidad ofrece una visión global cercana al tiempo real; la consistencia secuencial mantiene un orden común; la causal protege relaciones de dependencia; las garantías de sesión protegen la experiencia de un usuario; y la eventual permite retrasos temporales. No obstante, este orden es una simplificación, porque algunos modelos protegen propiedades diferentes y no siempre pueden compararse mediante una única escala [1], [3].

II-C. Teorema CAP

El teorema CAP fue demostrado formalmente por Gilbert y Lynch en 2002. Establece que, cuando ocurre una partición de red, un sistema distribuido no puede garantizar al mismo tiempo consistencia fuerte y disponibilidad completa [9]. Una partición ocurre cuando algunos nodos continúan funcionando, pero no pueden comunicarse entre ellos debido a una falla o interrupción de la red.

Dentro de CAP, la consistencia significa que todas las lecturas observan el valor más reciente, como si existiera una sola copia de los datos. La disponibilidad significa que

todo nodo que continúa funcionando debe responder a las solicitudes recibidas. La tolerancia a particiones indica que el sistema debe continuar operando aunque algunos mensajes entre nodos se pierdan o se retrasen [9].

El compromiso entre consistencia y disponibilidad aparece específicamente durante una partición de red. En esa situación, el sistema puede rechazar o retrasar algunas operaciones para evitar datos contradictorios, priorizando la consistencia, o puede continuar respondiendo en todos los nodos, priorizando la disponibilidad, aunque temporalmente se puedan utilizar datos desactualizados [9]. Por tanto, no es correcto interpretar CAP como una elección permanente de solamente dos propiedades; la decisión crítica aparece cuando los nodos dejan de comunicarse.

En el SCLI, CAP sería aplicable si la base de datos de reservas se replica en varios nodos. Por ejemplo, si dos réplicas pierden la comunicación y ambas reciben solicitudes para reservar el mismo laboratorio y horario, el sistema debe tomar una decisión. Puede impedir temporalmente que una de las réplicas confirme reservas, manteniendo la consistencia, o permitir que las dos continúen operando, aumentando la disponibilidad, pero con el riesgo de generar reservas duplicadas.

Para las reservas académicas resulta más apropiado priorizar la consistencia, debido a que una reserva duplicada afecta directamente la planificación del laboratorio. Esto significa que, si el sistema no puede comprobar de forma segura el estado actualizado de la agenda, debería rechazar o dejar pendiente la operación hasta recuperar la comunicación. Esta decisión reduce temporalmente la disponibilidad, pero protege la corrección de los datos.

También debe aclararse que una interrupción entre reservas-service y laboratorios-service no constituye por sí sola una aplicación formal del teorema CAP. CAP se relaciona directamente con el comportamiento de datos replicados durante una partición. La falla entre microservicios debe tratarse además mediante tiempos de espera, reintentos controlados, circuit breakers, idempotencia y mensajería asíncrona.

II-D. Principio PACELC

Abadi propuso PACELC para ampliar el análisis realizado por CAP. Su formulación indica que, si ocurre una partición de red, el sistema debe elegir entre disponibilidad y consistencia; de lo contrario, cuando la red funciona normalmente, debe elegir entre menor latencia y mayor consistencia [10]. Esta idea suele resumirse como: If Partition, Availability or Consistency; Else, Latency or Consistency.

PACELC muestra que las decisiones sobre consistencia no aparecen solamente durante una falla. Incluso cuando todos los nodos pueden comunicarse, garantizar una consistencia fuerte puede requerir que una operación espere la confirmación de varias réplicas. Esta coordinación aumenta el tiempo de respuesta, mientras que responder desde un solo nodo puede reducir la latencia, pero permitir temporalmente información desactualizada [10].

Aplicado al SCLI, una reserva puede aceptar un tiempo de respuesta ligeramente mayor para comprobar que ninguna otra

solicitud haya ocupado el mismo laboratorio y horario. En este caso, el sistema prioriza la consistencia sobre la latencia. La confirmación no debería enviarse hasta que la operación se registre de manera segura y se cumplan las reglas que impiden reservas duplicadas.

En cambio, los reportes, las métricas de monitoreo y algunas notificaciones pueden priorizar una respuesta rápida. Por ejemplo, un tablero podría tardar unos segundos en mostrar una reserva reciente sin afectar el funcionamiento principal del sistema. Para estos datos puede utilizarse consistencia eventual, porque un pequeño retraso resulta aceptable y la información terminará actualizándose.

Por lo tanto, PACELC permite evitar que todos los microservicios del SCLI utilicen el mismo nivel de consistencia. Las reservas y otras operaciones críticas deben priorizar datos correctos, aunque exista una latencia ligeramente mayor. Los reportes, la telemetría y ciertas notificaciones pueden priorizar velocidad y disponibilidad, siempre que finalmente alcancen un estado actualizado.

II-E. Estrategias de replicación

La replicación consiste en mantener copias del mismo conjunto de datos en diferentes nodos. Su propósito es mejorar la disponibilidad, facilitar la recuperación ante fallos y distribuir las consultas entre varios servidores. Su principal dificultad consiste en mantener actualizadas las copias cuando se modifica la información [1]. La replicación síncrona puede ofrecer garantías más fuertes, pero aumenta la comunicación y el tiempo de respuesta entre los nodos.

En la replicación líder-seguidor, un nodo principal recibe las operaciones de escritura y envía los cambios a uno o varios nodos secundarios. Esta organización facilita el control del orden de las modificaciones, porque las escrituras pasan por un punto principal. Si el líder falla, el sistema necesita promover uno de los seguidores para continuar aceptando cambios. Por lo tanto, el líder se convierte en un punto crítico únicamente cuando no existe un mecanismo adecuado de recuperación o cambio automático [4].

La replicación líder-seguidor puede funcionar de manera síncrona o asíncrona. En la modalidad síncrona, una operación espera la confirmación de una o varias réplicas seleccionadas antes de considerarse completada. Esto reduce el riesgo de perder una modificación confirmada, pero aumenta el tiempo de respuesta. En la modalidad asíncrona, el líder confirma primero la operación y envía el cambio posteriormente, lo que mejora la velocidad, aunque las réplicas pueden mostrar temporalmente información desactualizada [1].

La replicación multi-líder permite que varios nodos acepten escrituras. Esta estrategia puede ser útil cuando el sistema funciona en diferentes ubicaciones geográficas, porque cada región puede escribir en un nodo cercano. Su principal dificultad es resolver los conflictos que aparecen cuando dos líderes modifican el mismo dato al mismo tiempo [5]. GeoGauss constituye un ejemplo reciente de una base de datos SQL georeplicada con arquitectura multi-master y consistencia fuerte.

En una replicación sin líder no existe un nodo principal permanente. Las lecturas y escrituras se envían a varias réplicas y la operación se considera válida cuando responde una cantidad mínima de ellas, denominada quórum. Los quóruns de lectura y escritura deben cruzarse para que una lectura pueda encontrar las modificaciones confirmadas anteriormente [11].

Debe evitarse afirmar que una replicación por quórum siempre requiere un algoritmo de consenso. Un quórum establece cuántos nodos deben participar en una operación, mientras que un algoritmo como Raft o Paxos permite que los nodos acuerden un valor u orden común. Algunos sistemas utilizan ambos mecanismos, pero no son conceptos equivalentes. Los sistemas de quórum también presentan compromisos entre disponibilidad, latencia, tolerancia a fallos y carga de la red [11].

Para el SCLI, una configuración líder-seguidor sería una alternativa razonable si el sistema se despliega dentro de una sola institución. El nodo principal recibiría las escrituras y un nodo secundario mantendría una copia para recuperación. Una arquitectura multi-líder o sin líder introduciría mayor complejidad y solo estaría justificada si el sistema funcionara en varios centros de datos, tuviera una carga elevada o necesitara continuar aceptando escrituras ante la pérdida de una sede completa.

La arquitectura actual del SCLI define una base PostgreSQL independiente para cada microservicio, pero no especifica todavía réplicas, nodos secundarios ni mecanismos de elección de líder. Por tanto, el patrón Database per Service no debe presentarse como una estrategia de replicación. Las estrategias descritas en esta sección son alternativas que posteriormente deberán compararse con la implementación real del proyecto.

II-F. Consistencia entre microservicios

PostgreSQL puede aplicar las propiedades ACID dentro de la base de datos de cada microservicio. Esto permite que una operación local se complete totalmente o se revierta si ocurre un error. El problema aparece cuando un proceso necesita modificar bases de datos pertenecientes a varios servicios, porque una transacción local no puede controlar directamente todas esas bases independientes [2], [6]. Los estudios sobre gestión de datos en microservicios muestran que los procesos entre servicios se coordinan frecuentemente mediante flujos asíncronos y alcanzan consistencia eventual.

Antes de diseñar ese proceso en el SCLI, debe definirse con claridad qué servicio es responsable de cada dato. Una división apropiada sería que laboratorios-service controle el estado operativo del laboratorio, por ejemplo, activo, cerrado o en mantenimiento. Por su parte, reservas-service debería controlar la agenda y determinar si un horario ya fue reservado. Ambos servicios no deberían modificar una misma variable general de disponibilidad, porque se formarían dos fuentes de información que podrían contradecirse.

El patrón Saga permite coordinar una operación que pasa por varios microservicios. El proceso se divide en transacciones locales y cada servicio confirma únicamente los cambios

realizados en su propia base de datos. Cuando una operación termina, se envía un mensaje para iniciar el siguiente paso. Si una etapa falla, se ejecutan acciones compensatorias para contrarrestar las operaciones completadas anteriormente [6].

Una acción compensatoria no siempre equivale a restaurar exactamente el estado anterior. Por ejemplo, cancelar una reserva provisional puede compensar su creación, pero no borra el hecho de que la solicitud existió. Además, Saga no proporciona el mismo aislamiento que una transacción ACID, por lo que otros procesos pueden observar estados intermedios antes de que termine toda la operación [6].

En el SCLI, una Saga podría comenzar con la validación del estado operativo del laboratorio. Después, reservas-service comprobaría el horario y registraría la reserva. Finalmente, podrían generarse un evento de auditoría y una notificación. Si no es posible completar el registro de la reserva, el proceso debería quedar rechazado o pendiente; no tendría sentido modificar la agenda del laboratorio si la reserva principal no fue confirmada.

RabbitMQ permitiría transportar los eventos entre los servicios, pero el envío de mensajes no resuelve por sí solo la consistencia. La aplicación debe controlar mensajes repetidos, fallos temporales y operaciones incompletas. Las acciones deben diseñarse para que procesar nuevamente un mismo mensaje no produzca reservas duplicadas ni notificaciones incorrectas.

II-G. Síntesis del marco teórico

No existe una única estrategia de consistencia y replicación que sea adecuada para todos los datos. Las garantías fuertes facilitan el cumplimiento de reglas críticas, pero necesitan mayor coordinación y pueden aumentar la latencia. Las garantías más flexibles reducen el tiempo de respuesta y favorecen la disponibilidad, aunque permiten que algunas copias permanezcan desactualizadas temporalmente [1].

En el SCLI, la confirmación de una reserva debe utilizar controles fuertes para impedir que dos usuarios ocupen el mismo laboratorio y horario. En cambio, los reportes, las notificaciones, los paneles de monitoreo y las vistas de consulta de auditoría pueden actualizarse de manera eventual. El registro principal de una reserva y los eventos originales de auditoría deben conservarse correctamente, aunque su visualización en otros servicios aparezca algunos segundos después.

Esta conclusión todavía corresponde a una propuesta teórica. En el análisis del modelo implementado deberá aclararse que la arquitectura actual utiliza una instancia PostgreSQL independiente por servicio y que todavía no se ha incorporado una estrategia real de replicación.

III. ANÁLISIS DEL MODELO UTILIZADO EN EL PROYECTO

III-A. Estado actual del SCLI

El Sistema de Control de Laboratorios Informáticos se está desarrollando mediante una arquitectura de microservicios. Hasta el momento se han creado los servicios auth-service,

usuarios-service, academico-laboratorios-service y reservas-solicitudes-service. Esta organización concentra funciones relacionadas dentro de un mismo dominio y evita dividir el sistema en servicios demasiado pequeños.

El auth-service administra la autenticación, los roles, los permisos y la generación de tokens. El usuarios-service se encarga de los datos personales e institucionales de los usuarios. El academico-laboratorios-service reúne la información académica, los laboratorios y los equipos. Finalmente, el reservas-solicitudes-service concentra las solicitudes, aprobaciones, reservas, agenda y prevención de cruces de horarios.

Cada servicio está diseñado para administrar sus propios datos, siguiendo el patrón Database per Service. Esto significa que un microservicio no puede acceder directamente a las tablas de otro. Las relaciones entre dominios se representan mediante identificadores UUID y la información externa debe consultarse mediante APIs o intercambiarse mediante eventos. Esta separación forma parte de las reglas definidas para el proyecto.

III-B. Modelo implementado en el Auth Service

El servicio con mayor avance funcional es el auth-service. Actualmente permite iniciar sesión mediante nombre de usuario o correo electrónico, generar un JWT con una duración de quince minutos, proteger rutas y consultar la información del usuario autenticado mediante el endpoint GET /api/v1/auth/me.

También se encuentran implementados Spring Security, BCrypt, la validación de JWT, el filtro de autenticación y el manejo controlado de credenciales incorrectas. Además, se configuró CORS para que el frontend desarrollado con Vue pueda comunicarse con el servicio desde un origen diferente.

La base scli_auth contiene las tablas de usuarios de autenticación, roles, permisos, relaciones entre roles y permisos, refresh tokens e intentos de acceso. Las migraciones son administradas mediante Flyway, lo que permite crear y actualizar la estructura de manera controlada.

Todavía están pendientes la rotación de refresh tokens, el cierre de sesión, el bloqueo por intentos fallidos, el cambio de contraseña, la administración completa de credenciales y la integración definitiva con el API Gateway. Por tanto, estas funciones no deben presentarse en el informe como mecanismos ya operativos.

III-C. Consistencia entre microservicios

Cada microservicio del SCLI controla sus datos mediante transacciones locales. Estas transacciones permiten completar o revertir los cambios realizados dentro de una misma base, pero no pueden controlar automáticamente las bases de otros servicios. Por esta razón, una operación que involucra autenticación, usuarios, laboratorios y reservas no dispone de una única transacción global.

Al crear una reserva, el reservas-solicitudes-service debe comprobar que el laboratorio esté habilitado, validar que el horario continúe libre y registrar la operación. El estado

operativo del laboratorio pertenece al academico-laboratorios-service, mientras que la agenda y los cruces de horarios pertenecen al servicio de reservas. Esta separación evita que ambos servicios modifiquen el mismo dato y se conviertan en fuentes contradictorias.

Durante la primera etapa, la validación entre estos servicios se realizará mediante REST. Si el servicio académico no responde, la reserva no debería confirmarse, porque no existe evidencia suficiente de que el laboratorio esté habilitado. Este comportamiento reduce temporalmente la disponibilidad, pero evita registrar una operación incorrecta.

RabbitMQ todavía forma parte de la arquitectura propuesta y no de la implementación verificada. Cuando se incorpore, permitirá actualizar de manera eventual los servicios de auditoría, reportes y notificaciones. Por tanto, el sistema actualmente ofrece transacciones fuertes dentro de cada servicio, pero la consistencia eventual entre microservicios todavía debe implementarse.

III-D. Modelo de consistencia propuesto

El SCLI aplica un modelo híbrido. Las operaciones críticas requieren datos actualizados y controles fuertes, mientras que los procesos secundarios pueden aceptar pequeños retrasos.

La consistencia fuerte debe aplicarse al inicio de sesión, la asignación de roles, el cambio de contraseña, la aprobación de solicitudes, la confirmación de reservas y la prevención de cruces de horarios. En estos procesos, una lectura desactualizada podría permitir accesos indebidos o reservas duplicadas.

La consistencia eventual puede utilizarse en reportes, notificaciones, métricas y paneles de monitoreo. Por ejemplo, una reserva debe quedar registrada correctamente antes de responder al usuario, pero el correo de confirmación y la actualización del reporte pueden completarse unos segundos después.

Este modelo evita imponer el mismo costo de coordinación a todo el sistema. Las operaciones principales priorizan la corrección, mientras que las tareas informativas priorizan una respuesta rápida y un menor acoplamiento.

III-E. Uso propuesto de RabbitMQ

RabbitMQ permitirá que un microservicio publique eventos después de completar una operación. Por ejemplo, una reserva confirmada podría producir un evento ReservaCreada, que posteriormente sería procesado por auditoría, reportes y notificaciones.

El uso de mensajería evita que la reserva espere a que todos los servicios terminen su trabajo. No obstante, RabbitMQ no garantiza por sí solo la consistencia de las bases de datos. La aplicación debe controlar mensajes repetidos, reintentos, fallos de consumidores y eventos que no puedan procesarse.

Para reducir el riesgo de guardar una reserva sin publicar su evento, puede utilizarse el patrón Transactional Outbox. El cambio principal y el evento pendiente se almacenan en la misma transacción local. Después, un proceso separado envía el evento a RabbitMQ.

Este patrón introduce un compromiso: mejora la confiabilidad, pero requiere una tabla adicional, un publicador de eventos, limpieza de registros y consumidores idempotentes. Por tanto, agrega complejidad operativa a cambio de reducir la pérdida de mensajes.

III-F. Uso propuesto de Redis

Redis puede utilizarse como caché para catálogos, configuraciones, permisos y datos consultados con frecuencia. Esto reduce el número de accesos a las bases de datos y mejora el tiempo de respuesta.

Sin embargo, Redis no debe ser la fuente oficial de información. Los datos originales deben permanecer en la base administrada por cada microservicio. También será necesario definir tiempos de expiración e invalidación para impedir que la caché conserve información antigua.

En una reserva puede consultarse Redis para acelerar ciertas lecturas, pero la confirmación final debe ejecutarse en la base responsable de la agenda. El beneficio de una respuesta más rápida no justifica aceptar una reserva utilizando únicamente datos almacenados temporalmente.

El principal compromiso de Redis es que mejora el rendimiento, pero introduce otra copia del dato. Cuantas más copias existan, mayor será la dificultad para mantenerlas actualizadas.

III-G. Estrategia actual de replicación

El auth-service utiliza CockroachDB con aislamiento SERIALIZABLE, pero se ejecuta mediante start-single-node. Esto significa que actualmente existe una sola instancia y no hay réplicas coordinadas entre varios nodos.

El volumen de Docker conserva los datos cuando se reinicia el contenedor, pero no representa una réplica. Si el nodo deja de funcionar, el servicio de autenticación permanece indisponible hasta recuperar la base.

Por tanto, la implementación actual corresponde a una instancia única de CockroachDB con almacenamiento persistente, sin replicación y sin tolerancia real a particiones. Esta configuración es apropiada para desarrollo, pero no satisface todavía el objetivo de alta disponibilidad.

Una alternativa consiste en utilizar PostgreSQL con replicación primaria-réplica. El nodo primario recibe las escrituras y las réplicas mantienen copias mediante el registro WAL. PostgreSQL permite replicación síncrona o asíncrona, pero la promoción de una réplica, el balanceo y la recuperación deben configurarse mediante herramientas y procedimientos adicionales.

La segunda alternativa es desplegar CockroachDB como un clúster real. CockroachDB divide los datos en rangos y mantiene varias réplicas coordinadas mediante consenso. Para tolerar el fallo de un nodo se recomienda un mínimo de tres nodos, porque se necesita conservar una mayoría de réplicas disponible.

Estas opciones no deben mezclarse en una misma descripción. PostgreSQL utiliza normalmente un esquema primario-réplica, mientras que CockroachDB distribuye y replica automáticamente los rangos entre nodos. La elección debe de-

pender de los requisitos reales del SCLI y no únicamente de cuál tecnología parece más avanzada.

III-H. Análisis mediante CAP

En la configuración actual de un solo nodo, CAP no puede evaluarse completamente porque no existen réplicas que puedan quedar separadas. Si el nodo falla, el servicio simplemente deja de estar disponible.

CAP será relevante cuando se despliegue una base con varios nodos. Gilbert y Lynch establecen que, durante una partición, no pueden garantizarse simultáneamente consistencia fuerte y disponibilidad completa [9].

Para las reservas, un nodo aislado que no pueda comunicarse con la mayoría no debería confirmar operaciones. Esto reduce temporalmente la disponibilidad, pero evita que dos partes separadas acepten el mismo laboratorio y horario.

En los reportes y notificaciones puede tolerarse información temporalmente desactualizada. Sin embargo, esto no significa que un microservicio sea permanentemente CP o AP. La clasificación describe el comportamiento del almacenamiento replicado durante una partición concreta.

III-I. Análisis mediante PACELC

PACELC también analiza el sistema cuando la red funciona correctamente. Abadi explica que, en ausencia de una partición, debe decidirse entre una menor latencia y una mayor consistencia [10].

Al confirmar una reserva, el SCLI puede aceptar una demora adicional para validar el laboratorio, comprobar el horario y completar la transacción. En este proceso se prioriza la consistencia sobre la velocidad.

Los reportes, las notificaciones y las métricas pueden responder con los datos disponibles y actualizarse posteriormente. En estos procesos se prioriza una menor latencia y se acepta consistencia eventual.

Este compromiso debe evaluarse por operación. Aplicar consistencia fuerte a todos los datos aumentaría innecesariamente la coordinación, mientras que utilizar consistencia eventual en todo el sistema pondría en riesgo las operaciones críticas.

III-J. Trade-offs y análisis crítico de CockroachDB

CockroachDB ofrece consistencia serializable, replicación automática, distribución horizontal y tolerancia al fallo de nodos cuando funciona como clúster. Estas características son valiosas para operaciones concurrentes como reservas, autenticación y rotación de tokens. Sin embargo, sus ventajas aparecen realmente cuando se despliegan varios nodos; utilizarlo como nodo único conserva parte de su complejidad sin obtener alta disponibilidad.

El primer compromiso es el costo de infraestructura. Un despliegue tolerante al fallo de un nodo necesita al menos tres nodos, mientras que PostgreSQL puede comenzar con una instancia principal y añadir una réplica. Para un sistema institucional con carga moderada, un clúster distribuido puede consumir más memoria, almacenamiento y recursos administrativos de los necesarios.

El segundo compromiso es la latencia. En CockroachDB, una escritura distribuida debe alcanzar el quórum antes de confirmarse. Esto mejora la seguridad del dato, pero aumenta el tiempo de respuesta frente a una escritura local en PostgreSQL, especialmente cuando los nodos se encuentran en ubicaciones diferentes.

El tercer compromiso corresponde al manejo de concurrencia. CockroachDB utiliza SERIALIZABLE como nivel predeterminado y algunas transacciones pueden fallar con errores de reintento. La aplicación debe detectar estos casos y repetir únicamente operaciones seguras, lo que añade lógica y pruebas que no siempre son necesarias en una implementación tradicional de PostgreSQL.

El cuarto compromiso es la compatibilidad. CockroachDB utiliza el protocolo de PostgreSQL y soporta gran parte de su sintaxis, pero no es una copia completa de PostgreSQL. Algunas funciones, extensiones, procedimientos, cursores y características avanzadas presentan soporte parcial o comportamientos diferentes, por lo que una migración debe verificarse y no asumirse como directa.

El quinto compromiso es operativo. Un clúster exige administrar certificados, nodos, localidades, respaldos, actualizaciones, monitoreo, capacidad y recuperación ante fallos. PostgreSQL tiene una arquitectura más familiar y un ecosistema más amplio para equipos pequeños, aunque su escalamiento horizontal y su recuperación automática requieren componentes adicionales.

Frente a YugabyteDB, CockroachDB comparte objetivos como la distribución horizontal y la compatibilidad con PostgreSQL. La diferencia es que YugabyteDB expone la interfaz YSQL, mientras que CockroachDB utiliza su propio dialecto compatible con el protocolo PostgreSQL. Ambos introducen una curva de aprendizaje relacionada con clústeres, distribución y compatibilidad incompleta con determinadas funciones de PostgreSQL.

III-K. Postura crítica sobre el modelo elegido

CockroachDB no debe considerarse automáticamente la mejor opción para todos los microservicios del SCLI. Para una sola institución, con una carga moderada y un despliegue local, PostgreSQL puede resultar más sencillo, económico y suficiente. Una configuración primaria-réplica proporcionaría recuperación y escalamiento de lecturas sin introducir inicialmente toda la complejidad de una base distribuida.

CockroachDB sería más justificable en reservas-solicitudes-service si se demuestra una alta concurrencia, necesidad de escalamiento horizontal o continuidad ante el fallo de nodos. También podría mantenerse en auth-service si la autenticación requiere alta disponibilidad y el equipo está preparado para administrar un clúster real.

No sería recomendable utilizar PostgreSQL y CockroachDB de forma diferente en cada servicio sin una razón comprobable. La diversidad tecnológica aumenta el trabajo de respaldo, actualización, monitoreo y capacitación. La decisión debe apoyarse en pruebas de carga y requisitos de disponibilidad, no solo en las características promocionadas por cada producto.

III-L. Evaluación preliminar

El modelo actual es adecuado como base de desarrollo porque separa los dominios y mantiene consistencia fuerte dentro de cada microservicio. La agrupación de reservas y solicitudes en un mismo servicio también permite controlar los cruces mediante una transacción local.

No obstante, el modelo todavía no es óptimo para producción. CockroachDB funciona con un solo nodo, pero próximamente se planea una estrategia completa de replicación y consistencia eventual a los servicios.

La opción más razonable para esta etapa es conservar transacciones fuertes en las operaciones críticas, incorporar RabbitMQ de manera gradual para procesos secundarios y realizar pruebas antes de extender CockroachDB a todos los servicios. Si los resultados no justifican la complejidad del clúster, PostgreSQL con replicación primaria-réplica sería una alternativa más simple para el contexto institucional del SCLI.

IV. ESTADO ACTUAL DE LA ARQUITECTURA

La arquitectura del SCLI está diseñada para que el usuario acceda al sistema desde un frontend desarrollado con Vue.js. Las solicitudes pasarán primero por Nginx y luego por el API Gateway, que funcionará como punto único de entrada y se encargará de dirigir cada petición al microservicio correspondiente.

El sistema se divide por áreas de negocio. La arquitectura general contempla servicios independientes para autenticación, usuarios, gestión académica, laboratorios, reservas, solicitudes, auditoría, reportes, notificaciones y monitoreo. Esta separación permite que cada servicio tenga una responsabilidad concreta y pueda desarrollarse o desplegarse sin modificar todo el sistema.

En la implementación actual se encuentran en desarrollo cuatro componentes principales: auth-service, usuarios-service, academico-laboratorios-service y reservas-solicitudes-service. Los dominios académico y laboratorios, así como reservas y solicitudes, se han agrupado temporalmente en servicios conjuntos para facilitar el trabajo del equipo. La separación mostrada en la arquitectura representa la estructura objetivo que podrá adoptarse cuando el sistema aumente su tamaño.

El auth-service es el componente con mayor avance funcional. Actualmente permite iniciar sesión, generar tokens JWT, validar rutas protegidas y consultar al usuario autenticado. También dispone de roles, permisos, contraseñas cifradas con BCrypt y migraciones de base de datos administradas mediante Flyway.

La arquitectura aplica el patrón Database per Service. Esto significa que cada microservicio debe administrar su propia base de datos y no puede consultar directamente las tablas de otro servicio. Las relaciones externas se conservan mediante identificadores UUID y la información se intercambia a través de APIs.

Aunque el diagrama presenta PostgreSQL para todas las bases, actualmente el auth-service utiliza CockroachDB en

modo de un solo nodo. Esta configuración permite desarrollar y probar las transacciones, pero todavía no proporciona replicación ni alta disponibilidad. Los demás servicios podrán utilizar PostgreSQL o CockroachDB según los resultados del análisis técnico y las necesidades reales de cada dominio, como se puede ver en la Fig. 1.

La comunicación principal entre los servicios será síncrona mediante REST y JSON. Por ejemplo, el servicio de reservas podrá consultar al servicio de laboratorios para comprobar que un espacio se encuentre habilitado. RabbitMQ se incorporará para procesos que no necesitan una respuesta inmediata, como auditoría, reportes y notificaciones.

Redis se utilizará como caché para reducir consultas repetidas y mejorar el tiempo de respuesta. Sin embargo, no será la fuente oficial de los datos. La confirmación de operaciones críticas, como una reserva o la asignación de permisos, deberá realizarse directamente sobre la base de datos responsable.

La arquitectura también contempla Config Server para centralizar la configuración y Netflix Eureka para registrar y localizar los servicios. Estos componentes facilitarán que el Gateway encuentre dinámicamente cada microservicio, aunque su integración completa todavía forma parte de las siguientes etapas del desarrollo.

Para la observabilidad se han considerado Prometheus, Grafana y Zipkin. Prometheus recopilará métricas, Grafana permitirá visualizarlas y Zipkin mostrará el recorrido de una solicitud entre varios servicios. Estas herramientas ayudarán a identificar fallos, tiempos de respuesta elevados y problemas de comunicación.

Toda la solución será ejecutada mediante Docker Compose. Esto permitirá levantar los microservicios, las bases de datos, RabbitMQ, Redis y las herramientas de monitoreo dentro de una misma red de contenedores.

El SCLI cuenta actualmente con una base funcional y una arquitectura claramente definida. No obstante, todavía deben completarse la integración del API Gateway, Config Server, Eureka, RabbitMQ, Redis, la observabilidad y la replicación multinodo. Por esta razón, la Fig. 1 representa principalmente la arquitectura objetivo, mientras que la implementación actual corresponde a una primera etapa del sistema distribuido.

V. EVALUACIÓN DE DOS ALTERNATIVAS

La implementación actual del SCLI utiliza CockroachDB en modo de un solo nodo para el auth-service. Esta configuración permite trabajar con transacciones serializables y controlar operaciones concurrentes, pero no ofrece replicación real ni continuidad ante la caída del nodo. Por esta razón, se compara el modelo actual con dos alternativas: PostgreSQL con replicación primaria-réplica y CockroachDB desplegado como un clúster multinodo.

V-A. PostgreSQL con replicación primaria-réplica

La primera alternativa consiste en utilizar PostgreSQL con una instancia primaria encargada de las escrituras y una o varias réplicas que mantengan copias de la información. Las

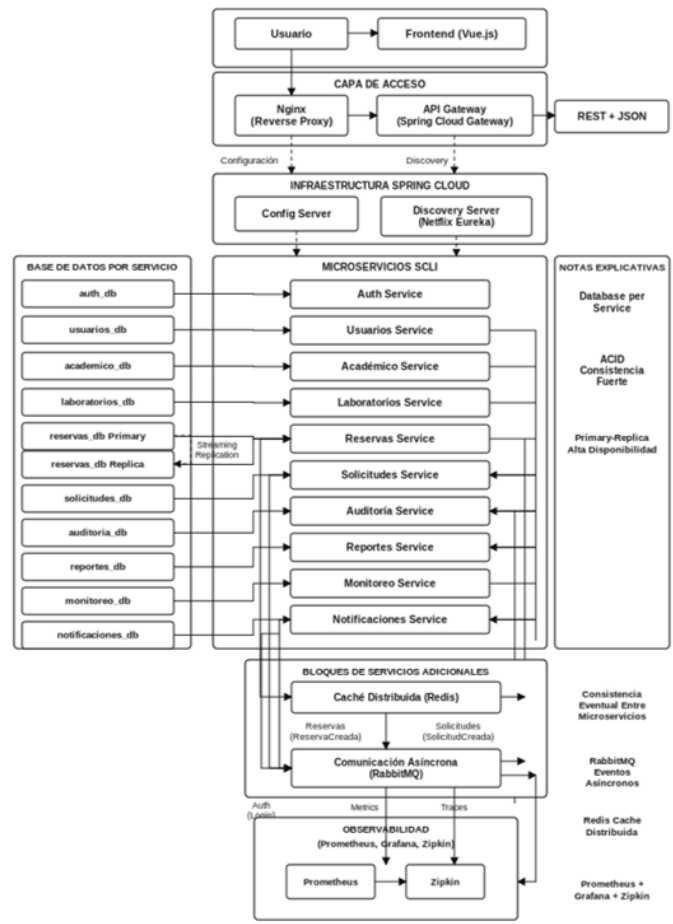


Figura 1: Arquitectura Distribuida SCLI.

réplicas pueden utilizarse para atender consultas o reemplazar al servidor principal cuando se produce un fallo.

Esta estrategia puede configurarse de forma síncrona o asíncrona. En la replicación síncrona, la operación espera la confirmación de una réplica antes de finalizar, lo que reduce el riesgo de perder cambios, pero aumenta el tiempo de respuesta. En la replicación asíncrona, el servidor principal confirma primero la operación y actualiza las réplicas después, por lo que se obtiene menor latencia a cambio de permitir un retraso temporal en las copias.

Para el SCLI, PostgreSQL con replicación primaria-réplica representa una alternativa adecuada para los servicios de usuarios, académico-laboratorios, auditoría, reportes y notificaciones. Estos servicios requieren una base relacional estable, pero no necesariamente necesitan distribuir las escrituras entre varios nodos. La evaluación de una arquitectura OLTP debe considerar conjuntamente el rendimiento, la disponibilidad, la durabilidad, la latencia y el costo de infraestructura, porque la solución más distribuida no siempre es la más conveniente para una carga concreta [12].

La principal ventaja de esta alternativa es su menor complejidad para un equipo pequeño. PostgreSQL posee un ecosistema amplio y es una tecnología conocida en el desarrollo

con Spring Boot. Sin embargo, la recuperación automática, la selección de un nuevo servidor primario y el balanceo de consultas requieren configuraciones o herramientas adicionales.

V-B. CockroachDB en clúster multinodo

La segunda alternativa consiste en desplegar CockroachDB como un clúster formado por varios nodos. En este modelo, la base divide la información en rangos y mantiene varias réplicas de cada rango. Las operaciones de escritura deben alcanzar el acuerdo de la mayoría de las réplicas antes de considerarse confirmadas [13].

CockroachDB ofrece transacciones distribuidas con aislamiento serializable, replicación automática y tolerancia al fallo de nodos. Estas características serían útiles para el *reservas-solicitudes-service*, donde dos solicitudes concurrentes no deben confirmar el mismo laboratorio y horario. También podrían beneficiar al *auth-service*, ya que la autenticación es un componente necesario para acceder al resto del sistema [13].

Esta alternativa tiene una mayor complejidad operativa. El equipo tendría que administrar varios nodos, certificados, respaldos, monitoreo, distribución de réplicas y procedimientos de recuperación. Las escrituras también pueden presentar mayor latencia, porque deben coordinarse entre diferentes nodos antes de ser confirmadas.

Por otra parte, las transacciones serializables pueden requerir reintentos cuando existen conflictos de concurrencia. Esto obliga a diseñar operaciones cortas e idempotentes y a controlar adecuadamente errores como SQLSTATE 40001. Por tanto, CockroachDB mejora la tolerancia a fallos, pero traslada parte de la complejidad hacia la infraestructura y la aplicación.

La comparación demuestra que ninguna alternativa es superior en todos los criterios. PostgreSQL primaria-réplica ofrece menor complejidad y costo, mientras que CockroachDB multinodo proporciona mejores mecanismos de distribución y recuperación. La decisión debe basarse en los requisitos reales de carga, disponibilidad y latencia del SCLI [12], [13].

VI. DISCUSIÓN CRÍTICA

El modelo actual es adecuado para el desarrollo inicial, pero no puede considerarse óptimo para producción. El uso de CockroachDB en un solo nodo permite probar las entidades, migraciones y transacciones del *auth-service*, pero elimina las principales ventajas de una base distribuida. Si el nodo deja de funcionar, la autenticación queda indisponible y los usuarios no pueden acceder al sistema.

Esto no significa que la decisión actual sea incorrecta. Durante el desarrollo, un solo nodo reduce el consumo de recursos y facilita la identificación de errores. El problema sería mantener esta configuración cuando el sistema entre en producción o presentarla como una arquitectura con replicación y alta disponibilidad.

PostgreSQL con replicación primaria-réplica representa la alternativa más equilibrada para una primera versión productiva del SCLI. Su menor complejidad resulta apropiada para un equipo de cuatro integrantes y para un sistema desplegado

inicialmente dentro de una sola institución. Además, permite mejorar la recuperación ante fallos sin incorporar desde el inicio toda la administración de un clúster distribuido.

Sin embargo, PostgreSQL tampoco elimina todos los problemas. Si la replicación es asíncrona, una réplica puede no contener las operaciones más recientes cuando falla el servidor principal. Si es síncrona, cada escritura debe esperar una confirmación adicional y aumenta la latencia. También deben configurarse mecanismos para detectar el fallo y promover una réplica. Estos compromisos confirman que el diseño de una arquitectura OLTP debe equilibrar costo, rendimiento, disponibilidad y durabilidad [12].

CockroachDB multinodo ofrece ventajas claras cuando la aplicación necesita continuar funcionando ante la pérdida de un nodo o distribuir una carga elevada. Su modelo de replicación y sus transacciones serializables pueden proteger operaciones críticas, como la creación concurrente de reservas [13].

No obstante, CockroachDB no debe seleccionarse únicamente porque sea una tecnología distribuida. Su adopción requiere más infraestructura y conocimientos sobre quóruns, distribución de datos, reintentos transaccionales y recuperación del clúster. En un proyecto con una carga institucional moderada, esta complejidad podría superar el beneficio obtenido.

Otro compromiso es la latencia. En un clúster distribuido, una escritura debe coordinarse con otras réplicas. Esto mejora la seguridad y disponibilidad del dato, pero normalmente tarda más que una escritura realizada en una única base local. Este efecto sería importante en el SCLI si los nodos se desplegaran en ubicaciones físicas diferentes.

También existe una curva de aprendizaje para el equipo. CockroachDB es compatible con el protocolo de PostgreSQL, pero su comportamiento distribuido obliga a comprender situaciones que no aparecen de la misma manera en una base tradicional, como los reintentos por conflictos serializables, la ubicación de réplicas y la pérdida de quórum. Taft *et al.* muestran que estas decisiones forman parte del funcionamiento interno necesario para ofrecer una base SQL geodistribuida y resiliente [13].

El uso de diferentes motores en cada microservicio también debe evaluarse con cautela. Utilizar PostgreSQL en unos servicios y CockroachDB en otros es posible, porque Database per Service permite elegir una tecnología según el dominio. Sin embargo, esta libertad aumenta el trabajo de despliegue, respaldo, actualización, monitoreo y capacitación. La gestión de datos en microservicios ya introduce una cantidad importante de lógica e infraestructura adicional, incluso sin combinar múltiples tecnologías [2].

La selección de la base de datos tampoco resuelve por sí sola la consistencia entre microservicios. Aunque el servicio de reservas utilice CockroachDB, una transacción local no puede modificar de forma automática las bases de usuarios, laboratorios, auditoría y notificaciones. Los estudios sobre microservicios identifican la coordinación de datos entre servicios independientes como uno de sus principales desafíos [2].

Cuadro I: Comparación de alternativas: PostgreSQL primaria-réplica y CockroachDB multinodo

Criterio	Modelo actual: CockroachDB de un nodo	PostgreSQL primaria-réplica	CockroachDB multinodo
Consistencia	Mantiene transacciones con aislamiento serializable dentro del único nodo, pero no coordina copias distribuidas [13].	El primario conserva la versión oficial del dato. La actualización observada en las réplicas depende de si la replicación es síncrona o asíncrona [1], [12].	Proporciona transacciones serializables y coordina las réplicas distribuidas antes de confirmar las operaciones [13].
Replicación	No existe replicación, porque se ejecuta con un solo nodo.	El primario transmite sus cambios a una o varias réplicas, síncrona o asíncronamente [1], [12].	Los datos se dividen en rangos y se replican automáticamente entre varios nodos [13].
Disponibilidad	Baja; el único nodo constituye un punto único de fallo.	Puede mejorar si existe una réplica preparada para sustituir al primario [12].	Alta mientras el clúster conserve la mayoría de réplicas necesaria [13].
Tolerancia a particiones	No puede evaluarse; solo existe un nodo.	Depende de la configuración de replicación y recuperación [1], [12].	Mantiene consistencia mediante consenso y evita escrituras de minorías aisladas [13].
Latencia de escritura	Generalmente menor, sin coordinación con otros nodos.	Menor con replicación asíncrona; mayor con síncrona [1], [12].	Puede ser mayor por el acuerdo de mayoría requerido [12], [13].
Escalabilidad	Principalmente vertical.	Distribuye lecturas entre réplicas; escrituras concentradas en el primario [12].	Distribución horizontal de datos y carga [13].
Complejidad operativa	Baja; una sola instancia.	Media; requiere replicación, supervisión y promoción [12].	Alta; nodos, réplicas, certificados, quóruns [13].
Costo de infraestructura	Bajo; un contenedor y volumen persistente.	Medio; primario y réplica [12].	Alto; varios nodos y mayores recursos [12], [13].
Principal riesgo	Indisponibilidad total si falla el nodo.	Retrasos o pérdida de cambios recientes; promoción incorrecta [1], [12].	Mayor latencia y complejidad; detiene escrituras sin mayoría [12], [13].
Aplicación propuesta	Adecuado solo para desarrollo y pruebas.	Alternativa razonable para primera versión productiva.	Apropiado si se demuestra necesidad de alta disponibilidad y escalamiento horizontal.

Para los procesos que abarcan varios servicios puede utilizarse el patrón Saga. Cada microservicio ejecuta una transacción local y, si una etapa falla, se realizan acciones compensatorias. Sin embargo, Saga no ofrece el mismo aislamiento que una transacción ACID y puede permitir que otros procesos observen estados intermedios [6].

En el SCLI no sería conveniente aplicar Saga al registro principal de una reserva si la solicitud, aprobación, agenda y reserva se encuentran dentro del mismo reservas-solicitudes-service. Es preferible proteger esos cambios mediante una transacción local. Saga o eventos asíncronos deben reservarse para las operaciones externas, como registrar auditoría, actualizar reportes o enviar notificaciones.

La principal limitación del modelo actual es la dependencia de un solo nodo. Como mitigación inicial se deben mantener respaldos verificados y procedimientos de restauración. Posteriormente, puede implementarse PostgreSQL primaria-réplica o un clúster piloto de CockroachDB para evaluar el comportamiento ante la caída de un nodo.

Otra limitación corresponde a las llamadas REST entre servicios. Si el servicio de reservas no puede consultar al servicio de laboratorios, la operación puede quedar bloqueada. Para reducir este riesgo se deben utilizar tiempos de espera, reintentos limitados, circuit breakers y estados pendientes. Una reserva no debería confirmarse si no puede comprobarse que el laboratorio está habilitado.

La futura comunicación con RabbitMQ también introduce el riesgo de mensajes duplicados o eventos no publicados. Esta

limitación puede mitigarse mediante Transactional Outbox, consumidores idempotentes, reintentos controlados y colas de mensajes fallidos. Estas medidas permiten alcanzar consistencia eventual sin depender de una transacción distribuida global [2], [6].

VI-A. Postura final

El modelo utilizado actualmente no es el óptimo para un entorno de producción, porque CockroachDB funciona en un solo nodo y todavía no existen replicación real, mensajería asíncrona ni recuperación automática. Sin embargo, sí resulta adecuado para la etapa actual de construcción y pruebas.

Para la primera versión productiva del SCLI, la alternativa más razonable es utilizar PostgreSQL con replicación primaria-réplica en los servicios de carga moderada. Esta opción ofrece un equilibrio entre consistencia, disponibilidad, costo y facilidad de administración.

CockroachDB multinodo debería evaluarse principalmente en el reservas-solicitudes-service y, de ser necesario, en el auth-service. Su adopción debe justificarse mediante pruebas de concurrencia, disponibilidad y recuperación ante fallos. Implementarlo en todos los microservicios sin evidencia de esa necesidad añadiría complejidad y costo sin garantizar un beneficio proporcional.

Por tanto, la estrategia recomendada es progresiva: mantener consistencia fuerte en las operaciones críticas, utilizar consistencia eventual para auditoría, reportes y notificaciones, implementar replicación según la importancia de cada servicio

y adoptar CockroachDB multinodo únicamente donde sus ventajas distribuidas sean realmente necesarias.

VII. CONCLUSIONES

El modelo de consistencia actualmente implementado en el SCLI no es todavía el óptimo para un entorno de producción, porque CockroachDB funciona en un solo nodo y no existe replicación real ni alta disponibilidad. Sin embargo, es adecuado para la etapa actual de desarrollo, ya que permite comprobar las transacciones y el funcionamiento de los microservicios con menor complejidad.

Para el dominio del SCLI, la opción más adecuada es un modelo híbrido. Las operaciones críticas, como la autenticación, los permisos, las solicitudes y las reservas, deben utilizar consistencia fuerte para evitar accesos incorrectos y cruces de horarios. En cambio, los reportes, las notificaciones y algunos registros de auditoría pueden trabajar con consistencia eventual, porque un pequeño retraso no afecta el proceso principal.

En una primera versión productiva, PostgreSQL con replicación primaria-réplica ofrece un equilibrio razonable entre disponibilidad, costo y facilidad de administración. CockroachDB multinodo sería conveniente únicamente si las pruebas demuestran que los servicios de autenticación o reservas requieren alta disponibilidad, escalamiento horizontal y tolerancia al fallo de nodos.

Por tanto, el modelo actual es válido como punto de partida, pero no constituye aún la solución óptima para el dominio del sistema. Para alcanzar ese nivel deberá incorporarse replicación real, manejo de fallos y comunicación asíncrona segura, manteniendo consistencia fuerte en los procesos principales y consistencia eventual en las tareas secundarias.

DECLARACIÓN DEL USO DE INTELIGENCIA ARTIFICIAL

Para la elaboración del presente informe se utilizó una herramienta de inteligencia artificial generativa como apoyo en la organización de ideas, la corrección de ambigüedades y la revisión gramatical. También se empleó para facilitar la explicación de conceptos relacionados con consistencia, replicación, CAP, PACELC y transacciones en bases de datos distribuidas.

La herramienta sirvió como apoyo para orientar la búsqueda de literatura científica y comparar alternativas como CockroachDB multinodo y PostgreSQL con replicación primaria-réplica.

La selección de las fuentes bibliográficas, la revisión de su pertinencia y la verificación de las referencias fueron realizadas manualmente por el autor. Cada cita utilizada en el informe fue comprobada de manera individual mediante la revisión de sus metadatos, DOI y contenido académico correspondiente. Para la consulta de los artículos científicos se utilizaron repositorios académicos y plataformas de acceso a publicaciones científicas, entre ellas ResearchGate, Sci-Hub y otros servicios de consulta disponibles para fines de revisión bibliográfica.

La interpretación de la información, el análisis comparativo entre las arquitecturas estudiadas, la aplicación al Sistema de Control de Laboratorios Informáticos y las conclusiones presentadas son responsabilidad exclusiva del autor del informe.

REFERENCIAS

- [1] R. A. Campêlo, M. A. Casanova, D. O. Guedes, and A. H. F. Laender, "A brief survey on replica consistency in cloud environments," *Journal of Internet Services and Applications*, vol. 11, Art. no. 1, 2020, doi: 10.1186/s13174-020-0122-y.
- [2] R. N. Laigner, Y. Zhou, M. A. V. Salles, Y. Liu, and M. Kalinowski, "Data management in microservices: State of the practice, challenges, and research directions," *Proceedings of the VLDB Endowment*, vol. 14, no. 13, pp. 3348–3361, 2021, doi: 10.14778/3484224.3484232.
- [3] R. Zennou, R. Biswas, A. Bouajjani, C. Enea, and M. Erradi, "Checking causal consistency of distributed databases," *Computing*, vol. 104, no. 10, pp. 2181–2201, 2022, doi: 10.1007/s00607-021-00911-3.
- [4] F. Han *et al.*, "PALF: Replicated write-ahead logging for distributed databases," *Proceedings of the VLDB Endowment*, vol. 17, no. 12, pp. 3745–3758, 2024, doi: 10.14778/3685800.3685803.
- [5] W. Zhou *et al.*, "GeoGauss: Strongly consistent and light-coordinated OLTP for geo-replicated SQL database," *Proceedings of the ACM on Management of Data*, vol. 1, no. 1, Art. no. 62, pp. 1–27, 2023, doi: 10.1145/3588916.
- [6] E. Daraghmi, C.-P. Zhang, and S.-M. Yuan, "Enhancing saga pattern for distributed transactions within a microservices architecture," *Applied Sciences*, vol. 12, no. 12, Art. no. 6242, 2022, doi: 10.3390/app12126242.
- [7] C. Araújo, M. Oliveira Jr., B. Nogueira, P. Maciel, and E. Tavares, "Performativity evaluation of NoSQL-based storage systems," *Journal of Systems and Software*, vol. 208, Art. no. 111885, Feb. 2024, doi: 10.1016/j.jss.2023.111885.
- [8] C. Camilleri, J. G. Vella, and V. Nezval, "Horizontally scalable implementation of a distributed DBMS delivering causal consistency via the actor model," *Electronics*, vol. 13, no. 17, Art. no. 3367, 2024, doi: 10.3390/electronics13173367.
- [9] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, Jun. 2002, doi: 10.1145/564585.564601.
- [10] D. J. Abadi, "Consistency tradeoffs in modern distributed database system design: CAP is only part of the story," *Computer*, vol. 45, no. 2, pp. 37–42, Feb. 2012, doi: 10.1109/MC.2012.33.
- [11] M. Whittaker, A. Charapko, J. M. Hellerstein, H. Howard, and I. Stoica, "Read-write quorum systems made practical," in *Proc. 8th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC '21)*, Online, United Kingdom, Apr. 2021, pp. 1–8, doi: 10.1145/3447865.3457962.
- [12] M. Haubenschild and V. Leis, "OLTP in the cloud: Architectures, tradeoffs, and cost," *The VLDB Journal*, vol. 34, Art. no. 42, 2025, doi: 10.1007/s00778-025-00913-z.
- [13] R. Taft *et al.*, "CockroachDB: The resilient geo-distributed SQL database," in *Proc. 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*, 2020, pp. 1493–1509, doi: 10.1145/3318464.3386134.